

IRepository<T>

```
/// <summary>
/// 레포지토리 인터페이스
/// </summary>
/// <typeparam name="T"> </typeparam>
public interface IRepository<T> where T : class, new()
{
    /// <summary>
    /// 특정 ID에 해당하는 엔티티를 비동기적으로 가져옵니다.
    /// </summary>
    /// <param name="id">검색하려는 엔티티의 고유 식별자(ID)</param>
    /// <returns>
    /// 요청된 ID에 해당하는 엔티티 객체를 반환합니다.
    /// 만약 해당 ID의 엔티티가 없으면 null을 반환합니다.
    /// </returns>
    Task<T?> GetByIdAsync(int id);

    /// <summary>
    /// 모든 엔티티를 비동기적으로 가져옵니다.
    /// </summary>
    /// <returns>
    /// 데이터베이스에 존재하는 모든 엔티티를 컬렉션 형태로 반환합니다.
    /// 엔티티의 목록을 IEnumerable 반환하며, 비어 있을 수 있습니다.
    /// </returns>
    Task<IEnumerable<T>> GetAllAsync();

    /// <summary>
    /// 새로운 엔티티를 비동기적으로 추가합니다.
    /// </summary>
    /// <param name="entity">추가할 엔티티 객체</param>
    /// <remarks>
    /// 새로 추가하려는 엔티티는 유효한 값이어야 하며,
    /// 해당 엔티티의 기본 키(ID)는 자동으로 생성되거나 지정되어야 합니다.
    /// </remarks>
    Task CreateAsync(T entity);
```

```

/// <summary>
/// 기존 엔티티를 비동기적으로 업데이트합니다.
/// </summary>
/// <param name="entity">수정할 엔티티 객체</param>
/// <remarks>
/// 주어진 엔티티 객체는 반드시 이미 데이터베이스에 존재하는 엔티티이어야 하며,
/// 해당 엔티티의 식별자는 변경되지 않아야 합니다.
/// 만약 엔티티가 존재하지 않으면 예외가 발생할 수 있습니다.
/// </remarks>
Task UpdateAsync(T entity);

```

```

/// <summary>
/// 엔티티를 비동기적으로 삭제합니다.
/// </summary>
/// <param name="entity">삭제할 엔티티 객체</param>
/// <remarks>
/// 삭제하려는 엔티티는 반드시 데이터베이스에 존재하는 엔티티여야 합니다.
/// 만약 해당 엔티티가 데이터베이스에 존재하지 않으면 예외가 발생할 수 있습니다.
/// 삭제 후, 엔티티는 더 이상 데이터베이스에 존재하지 않게 됩니다.
/// </remarks>
Task RemoveAsync(T entity);
}

```

where T : class, new() -> 제네릭 Type 제약을 정의하는 구문입니다.
이 구문은 제네릭 타입 파라미터 T 에 대해 특정한 제한을 두어, 해당 제네릭 Type 이 특정 조건을 만족하도록 합니다. T -> 제네릭 Type 입니다.

class 제약은 T 가 클래스 타입이어야 합니다.

예를 들어 T 는 int 같은 값 타입이나 struct(구조체)를 사용할 수 없습니다.

대신 T 는 string, MyClass, List<T>와 같은 클래스 타입이어야 합니다.

new() 는 디폴트 생성자가 존재하는 클래스 타입에 대한 제약입니다.

T 는 기본 생성자를 통해 인스턴스를 생성할 수 있는 타입이어야 한다는 조건입니다.

이 조건은 T 가 인수 없이 인스턴스를 생성할 수 있는 생성자를 제공하는 타입이어야 함을 의미합니다. 즉 T 는 매개변수가 없는 생성자가 있어야 합니다.

```

public class Repository<T> : IRepository<T> where T : class, new()
{

```

```

    // 데이터베이스 컨텍스트 객체 (DbContext)

```

```

    private readonly DbContext _context;

```

// 해당 엔티티 타입에 대한 DbSet 객체 (EF Core에서 테이블을 나타냄)

```
private readonly DbSet<T> _dbSet;
```

/// <summary>

/// Repository의 생성자. DbContext와 DbSet을 받아 초기화합니다.

/// </summary>

/// <param name="context">DbContext 객체 (데이터베이스 연결 및 트랜잭션 관리)</param>

/// <param name="dbSet">현재 엔티티에 해당하는 DbSet 객체 (EF Core에서 테이블로 매핑됨)</param>

```
public Repository(DbContext context, DbSet<T> dbSet)
```

```
{
```

```
    _context = context; // DbContext 초기화
```

```
    _dbSet = dbSet;     // 해당 엔티티 타입의 DbSet 초기화
```

```
}
```

/// <summary>

/// 새로운 엔티티를 데이터베이스에 비동기적으로 추가합니다.

/// </summary>

/// <param name="entity">추가할 엔티티 객체</param>

```
public async Task CreateAsync(T? entity)
```

```
{
```

```
    // 엔티티가 null이 아닐 경우에만 추가
```

```
    if (entity == null) throw new ArgumentNullException(nameof(entity));
```

```
    // 비동기적으로 엔티티를 DbSet에 추가
```

```
    await _dbSet.AddAsync(entity);
```

```
    // 데이터베이스에 변경 사항을 저장
```

```
    await _context.SaveChangesAsync();
```

```
}
```

/// <summary>

/// 모든 엔티티를 비동기적으로 가져옵니다.

/// </summary>

/// <returns>모든 엔티티를 담은 컬렉션 (IEnumerable<T>)</returns>

```
public async Task<IEnumerable<T>> GetAllAsync()
```

```
{
```

```
    // DbSet에서 모든 엔티티를 비동기적으로 가져옴
```

```
    return await _dbSet.ToListAsync();
```

```
}
```

```
/// <summary>
```

```
/// 특정 ID에 해당하는 엔티티를 비동기적으로 가져옵니다.
```

```
/// </summary>
```

```
/// <param name="id">검색하려는 엔티티의 고유 ID</param>
```

```
/// <returns>찾은 엔티티 또는 null</returns>
```

```
public async Task<T?> GetByIdAsync(int id)
```

```
{
```

```
    // 주어진 ID에 해당하는 엔티티를 비동기적으로 찾음
```

```
    return await _dbSet.FindAsync(id);
```

```
}
```

```
/// <summary>
```

```
/// 기존 엔티티를 데이터베이스에서 비동기적으로 삭제합니다.
```

```
/// </summary>
```

```
/// <param name="entity">삭제할 엔티티 객체</param>
```

```
public async Task RemoveAsync(T? entity)
```

```
{
```

```
    // 엔티티가 null일 경우 예외를 발생시킴
```

```
    if (entity == null) throw new ArgumentNullException(nameof(entity));
```

```
    // 엔티티를 DbSet에서 제거
```

```
    _dbSet.Remove(entity);
```

```
    // 데이터베이스에 변경 사항을 저장
```

```
    await _context.SaveChangesAsync();
```

```
}
```

```
/// <summary>
```

```
/// 기존 엔티티를 데이터베이스에서 비동기적으로 업데이트합니다.
```

```
/// </summary>
```

```
/// <param name="entity">수정할 엔티티 객체</param>
```

```
public async Task UpdateAsync(T? entity)
```

```
{
```

```
    // 엔티티가 null일 경우 예외를 발생시킴
```

```
    if (entity == null) throw new ArgumentNullException(nameof(entity));
```

```
// 엔티티를 DbSet에서 업데이트
_dbSet.Update(entity);

// 데이터베이스에 변경 사항을 저장
await _context.SaveChangesAsync();
}
}
```

Repository<T>

```
/// <summary>
/// 제네릭 타입 매개변수 <c>T</c>에 대한 제약 조건을 설명합니다.
/// <para><c>where T : class, new()</c> 구문은 <c>T</c>가
/// 다음 조건을 만족하도록 제한합니다:</para>
///
/// <list type="bullet">
///   <item><description><c>class</c>: <c>T</c>는 참조 형식이어야 하며,
///   값 형식(int, struct 등)은 허용되지 않습니다.</description></item>
///   <item><description><c>new()</c>: <c>T</c>는 매개변수가 없는
///   기본 생성자를 가져야 하며, 이를 통해 인스턴스화할 수 있어야 합니다.
///   </description></item>
/// </list>
///
/// <para>예: <c>T</c>는 <c>string</c>, 사용자 정의 클래스(<c>MyClass</c>),
/// <c>List<T></c> 등 참조형 클래스여야 하며, 기본 생성자가 필요합니다.</para>
/// </summary>
/// <typeparam name="T">기본 생성자가 있는 참조형 타입</typeparam>
```

```
public class Repository<T> : IRepository<T> where T : class, new()
{
    // Database 컨텍스트 개체 (DbContext) dbcontext 클래스명 예: AppDbContext.cs 파일
    private readonly DbContext _context;

    // 해당 엔티티 타입에 대한 DbSet 개체 (EF Core에서 테이블을 나타냄)
    private readonly DbSet<T> _dbSet;

    /// <summary>
    /// Repository 생성자. DbContext를 주입받아 내부 DbSet을 초기화합니다.
    /// </summary>
    /// <param name="context">EF Core DbContext 객체</param>
    public Repository(DbContext context)
    {
        // DbContext 초기화
        _context = context ?? throw new ArgumentNullException(nameof(context));
    }
}
```

```

// 해당 엔티티 타입의 DbSet 초기화
_dbSet = _context.Set<T>();
}

/// <summary>
/// 데이터베이스에 존재하는 모든 엔티티를 비동기적으로 가져옵니다.
/// </summary>
/// <returns>
/// 모든 엔티티를 담은 <see cref="IEnumerable{T}" /> 컬렉션입니다.
/// 결과가 없을 경우 빈 컬렉션이 반환됩니다.
/// </returns>
public async Task<IEnumerable<T>> GetAllAsync()
{
    // DbSet에서 모든 엔티티를 비동기적으로 가져옴
    return await _dbSet.ToListAsync();
}

/// <summary>
/// 특정 ID에 해당하는 엔티티를 비동기적으로 가져옵니다.
/// </summary>
/// <param name="id">검색하려는 엔티티의 고유 ID</param>
/// <returns>찾은 엔티티 또는 null</returns>
public async Task<T?> GetByIdAsync(int id)
{
    // 주어진 ID에 해당하는 엔티티를 비동기적으로 찾음
    return await _dbSet.FindAsync(id);
}

/// <summary>
/// 새로운 엔티티를 데이터베이스에 비동기적으로 추가합니다.
/// </summary>
/// <param name="entity">추가할 엔티티 객체</param>
public async Task CreateAsync(T entity)
{
    // 엔티티가 null이 아닐 경우에만 추가
    if (entity == null)
    {

```

```

        throw new ArgumentNullException(nameof(entity));
    }

    // 비동기적으로 엔티티를 DbSet에 추가
    await _dbSet.AddAsync(entity);

    // 데이터베이스에 변경 사항을 저장
    await _context.SaveChangesAsync();
}

/// <summary>
/// 기존 엔티티를 데이터베이스에서 비동기적으로 업데이트합니다.
/// </summary>
/// <param name="entity">수정할 엔티티 객체</param>
public async Task UpdateAsync(T entity)
{
    // 엔티티가 null일 경우 예외를 발생시킴
    if (entity == null)
    {
        throw new ArgumentNullException(nameof(entity));
    }

    // 엔티티를 DbSet에서 업데이트
    _dbSet.Update(entity);

    // 데이터베이스에 변경 사항을 저장
    await _context.SaveChangesAsync();
}

/// <summary>
/// 기존 엔티티를 데이터베이스에서 비동기적으로 삭제합니다.
/// </summary>
/// <param name="entity">삭제할 엔티티 객체</param>
public async Task RemoveAsync(T entity)
{
    // 엔티티가 null일 경우 예외를 발생시킴
    if (entity == null)
    {

```



```
        throw new ArgumentNullException(nameof(entity));
    }

    // 엔티티를 DbSet에서 제거
    _dbSet.Remove(entity);

    // 데이터베이스에 변경 사항을 저장
    await _context.SaveChangesAsync();
}
}
```

InMemoryRepository<T>

```
/// <summary>
/// 제네릭 인터페이스 <c>IRepository<T></c>
/// 를 구현한 메모리 기반 저장소 클래스입니다.
/// <typeparam name="T">클래스 타입이며,
/// 기본 생성자(<c>new()</c>)가 필요합니다.</typeparam>
/// </summary>
public class InMemoryRepository<T> : IRepository<T> where T : class, new()
{
    // 메모리에 데이터를 저장할 내부 리스트
    private readonly List<T> _items = new();

    /// <summary>
    /// 데이터를 저장소에 추가합니다.
    /// </summary>
    /// <param name="entity">추가할 엔티티</param>
    /// <returns></returns>
    public Task CreateAsync(T entity)
    {
        // 리스트에 엔티티 추가
        _items.Add(entity);

        // 비동기 작업 완료를 나타냄 (실제 작업 없음)
        return Task.CompletedTask;
    }

    /// <summary>
    /// 저장소에 저장된 모든 엔티티를 반환합니다.
    /// </summary>
    /// <returns>저장된 엔티티 목록</returns>
    public Task<IEnumerable<T>> GetAllAsync() =>
        Task.FromResult(_items.AsEnumerable());

    /// <summary>
    /// ID 값을 기반으로 특정 엔티티를 검색합니다.
    /// </summary>
```

```

/// <param name="id">검색할 ID 값</param>
/// <returns>ID에 해당하는 엔터티 (없으면 null)</returns>
public Task<T?> GetByIdAsync(int id)
{
    // 리플렉션을 사용하여 T 타입의 'id' 프로퍼티를 가져옴
    var prop = typeof(T).GetProperty("Id");

    if (prop == null)
        throw new InvalidOperationException("T Type 은 Type int의 'Id' 속성을 가져야 합니다.");

    // 'id' 프로퍼티가 존재하고, 해당 값을 비교하여 일치하는 첫 번째 항목을 찾음
    return Task.FromResult(_items.FirstOrDefault(x => (int)prop?.GetValue(x)! == id));
}

/// <summary>
/// 저장소에서 특정 엔터티를 제거합니다.
/// </summary>
/// <param name="entity">제거할 엔티티</param>
/// <returns></returns>
public Task RemoveAsync(T entity)
{
    // 리스트에서 해당 엔터티 제거
    _items.Remove(entity);
    return Task.CompletedTask;
}

/// <summary>
/// 기존 엔터티를 ID를 기준으로 찾아 업데이트합니다.
/// </summary>
/// <param name="entity"></param>
/// <returns></returns>
public Task UpdateAsync(T entity)
{
    // 'Id' 프로퍼티를 가져옴
    var prop = typeof(T).GetProperty("Id");
    if (prop == null)
        throw new InvalidOperationException("T Type 은 Type int의 'Id' 속성을 가져야 합니다.");

```

```
// 전달된 entity의 ID 값을 가져옴
```

```
int id = (int)prop.GetValue(entity)!;
```

```
// 기존 엔터티를 리스트에서 찾아 인덱스를 얻음
```

```
var index = _items.FindIndex(x => (int)prop.GetValue(x)! == id);
```

```
if (index >= 0)
```

```
{  
    _items[index] = entity; // 해당 위치의 항목을 새 엔터티로 교체  
}
```

```
// 실제 비동기 작업은 없지만 Task 반환이 필요한 경우
```

```
// 진짜 비동기 처리는 없지만 async 시그니처가 필요한 경우
```

```
return Task.CompletedTask;
```

```
}
```

```
}
```